1 fMRI for Program Comprehension

Motivation: Indirect measures (performance, surveys) fail to capture actual cognition in code understanding. Methodology: fMRI used to observe brain activity while developers perform comprehension tasks with controlled baselines. Key Findings: Code comprehension activates left-hemisphere regions tied to working memory, attention, and language, indicating a hybrid cognitive process distinct from pure math or language. Implications: Enables objective measurement of cognitive load, allowing evaluation of languages, tools, expertise, and code complexity.

2 Code Readability and Cognitive Load

Motivation: Poor naming and readability are suspected to increase effort but lack strong empirical validation. Methodology: Controlled experiments with varied identifier quality and readability, measuring task accuracy, time, and physiological cognitive load. Key Findings: Low-quality identifiers and poor structure significantly increase mental effort; combined effects further degrade comprehension and performance. Implications: Readability is a core quality attribute; enforcing naming conventions and clean structure directly improves developer efficiency.

3 Code Review and Expertise in the Brain

Motivation: It is unclear whether code is processed like natural language or as a distinct cognitive task. Methodology: fMRI experiments comparing code comprehension, code review, and prose reading, with machine learning classification of neural patterns. Key Findings: Code tasks are distinguishable from prose; however, experts show more overlap, suggesting abstraction and familiarity. Implications: Programming is a unique cognitive domain, and expertise changes mental representations, impacting education and tool design.

4 Learning a New Programming Language

Motivation: Developers struggle with new languages despite prior experience due to transfer of incorrect assumptions. Methodology: Mixed-method study combining Stack Overflow analysis and developer interviews to identify learning patterns. Key Findings: Cross-language interference is common; developers rely on opportunistic, just-in-time learning, leading to shallow or incorrect mental models. Implications: Learning tools should explicitly address differences between languages and support conceptual transitions, not just syntax learning.

5 Predictors of Developer Productivity

Motivation: Productivity is often mismeasured, with unclear drivers across environments. Methodology: Large-scale survey with regression analysis evaluating technical, social, and organizational factors. Key Findings: Social factors (motivation, peer support, feedback) consistently outweigh technical factors; interruptions and environment strongly influence perceived productivity. Implications: Improving productivity requires better team culture, communication, and workflow design, not just better tools.

6 Developer Productivity and Code Quality at Google

Motivation: Prior work shows correlations but lacks causal evidence for productivity drivers. Methodology: Longitudinal analysis combining surveys and engineering logs using fixed-effects and lagged regression models. Key Findings: Code quality is the strongest predictor of future productivity; improvements in code health precede efficiency gains; tool satisfaction and stable priorities also matter. Implications: Investing in maintainability and reducing technical debt yields immediate productivity benefits.

7 Logs-Based Focus and Flow

Motivation: Flow is important but difficult to measure without intrusive self-reporting. Methodology: Multi-phase study using developer logs, embeddings of actions, and validation against surveys to derive a behavioral focus metric. Key Findings: Pure flow cannot be directly inferred, but focused work can be detected; frequency of focus sessions correlates strongly with perceived productivity. Implications: Provides scalable, passive measurement of focus, enabling organizations to study interruptions and optimize work patterns.

8 GenAI, Creativity, and Software Development

Motivation: Research focuses on productivity gains from AI, while its effect on creativity is underexplored. Methodology: Conceptual analysis using the 4P creativity framework and McLuhan's tetrad to explore potential impacts. Key Findings: AI enhances ideation and reduces routine effort but risks homogenization, reduced originality, and over-reliance on generated solutions. Implications: Calls for research to ensure AI augments human creativity and supports diverse, novel problem-solving approaches.

9 Beliefs, Practices, and Personalities of Engineers

Motivation: The relationship between personality traits, engineering practices, and beliefs is not well understood. Methodology: Large-scale industrial survey using the Five-Factor Model along with questions on practices and preferences. Key Findings: Strong agreement on practices like testing and readability; disagreement on tools and environments; minimal personality differences across roles, with slight variation for managers. Implications: Engineering culture is broadly shared, and effectiveness depends more on context and practices than inherent personality differences.

10 Requirements Elicitation Survey

Motivation: Requirements elicitation is critical yet complex due to stakeholder diversity, ambiguity, and evolving needs. Methodology: Comprehensive survey categorizing elicitation techniques (interviews, questionnaires, observation, prototyping) and approaches across contexts. Key Findings: No single technique is sufficient; effectiveness depends on project type, stakeholder availability, and domain complexity; hybrid approaches are most common. Implications: Practitioners should select techniques contextually and combine methods; tool support and structured processes are essential to improve elicitation quality.

11 RE Techniques and Approaches Comparison

Motivation: Practitioners struggle to choose appropriate elicitation techniques due to lack of clear comparisons. Methodology: Comparative analysis of techniques based on cost, effectiveness, scalability, and stakeholder involvement. Key Findings: Interviews provide depth but are costly; surveys scale but lack richness; observation captures real behavior but is time-intensive; prototyping improves clarity but may bias requirements. Implications: Trade-offs must be considered explicitly; combining techniques mitigates individual weaknesses and improves requirement completeness.

12 Tool Support in Requirements Engineering

Motivation: Manual elicitation processes are error-prone and difficult to scale. Methodology: Survey of tools supporting documentation, collaboration, traceability, and automation in RE workflows. Key Findings: Tools improve organization and traceability but cannot replace human judgment; adoption depends on usability and integration with workflows. Implications: Effective RE requires both tool support and human-centered processes; future tools should better integrate automation (e.g., AI) with stakeholder interaction.

13 Issues and Pitfalls in Requirements Engineering

Motivation: RE failures often lead to project delays and defects, yet common pitfalls persist. Methodology: Analysis of recurring issues across studies and industry reports. Key Findings: Ambiguity, incomplete requirements, communication gaps, and stakeholder misalignment are dominant issues; over-reliance on a single technique worsens outcomes. Implications: Emphasizes need for iterative elicitation, validation, and stakeholder engagement throughout development.

14 Trends and Challenges in RE

Motivation: Software systems are becoming more complex, requiring evolution in RE practices. Methodology: Survey of emerging trends and open research challenges. Key Findings: Increasing importance of agile RE, continuous elicitation, and integration with AI-driven tools; challenges include handling dynamic requirements and large-scale stakeholder input. Implications: RE must become more adaptive, data-driven, and tool-supported to remain effective in modern development environments.

15 Conversational AI in RE Education

Motivation: AI tools like ChatGPT are increasingly used in education, but their role in RE learning is unclear. Methodology: Case study/course setup where students use conversational AI for RE tasks such as requirement drafting and refinement. Key Findings: AI assists in idea generation and structuring requirements but may introduce inaccuracies or superficial understanding; students rely on AI for efficiency. Implications: AI can enhance learning but must be guided with critical evaluation; educators should design tasks that balance assistance with deep understanding.

16 AI-Assisted Requirements Engineering

Motivation: AI has potential to automate parts of RE, but its effectiveness and risks are not fully understood. Methodology: Empirical and conceptual analysis of AI-supported RE tasks such as requirement extraction, summarization, and validation. Key Findings: AI improves speed and coverage but may introduce hallucinations, bias, and loss of context; human oversight remains essential. Implications: RE workflows should incorporate AI as an assistive tool rather than a replacement, with validation mechanisms to ensure correctness.

17 Human-AI Collaboration in Software Engineering

Motivation: AI tools are transforming development, but their broader impact on workflows and cognition is still emerging. Methodology: Synthesis of empirical studies and conceptual frameworks analyzing human-AI interaction in development tasks. Key Findings: AI improves productivity and exploration but affects reasoning, trust, and reliance patterns; developers shift from writing to supervising code. Implications: Future tools must support transparency, trust, and collaboration, ensuring developers remain engaged in reasoning rather than passive consumers.

18 AI, Productivity, and Developer Behavior

Motivation: Claims of productivity gains from AI tools require deeper understanding of behavioral changes. Methodology: Analysis of studies comparing developers using AI vs traditional workflows. Key Findings: AI increases speed and reduces effort for routine tasks, but may increase errors or reduce deep comprehension; novice vs expert usage differs significantly. Implications: Productivity gains must be balanced with quality and learning; AI tools should adapt to user expertise and task complexity.